

HRC2026 TRAINING CAMP

TASK 1 TEAM 4 GUIDE

Hướng dẫn chi tiết cho Người 1: Camera/RGB-D + Perception

Tài liệu nội bộ của BA x AILab

Hạng mục	Nội dung
Phiên bản	v1.0 FULL
Đối tượng	Tài+Hào+Trình: phụ trách Camera/RGB-D + Perception trong nhóm 4 người giải Task 1
Mục tiêu cuối	Tạo được perception module xuất ObjectState đủ sạch để Planner và Motion gấp được vật
Task	Task 1 - Precise Desktop Sorting of Workpieces
Output chính	sample_rgb.png, sample_depth.npy, depth_sanity_report.md, camera_config_sheet.csv, overlay_detection.png, confusion_matrix_task1.csv, pose_estimator_report.csv, yaw_report.csv, perception_interface.json
Thông điệp chính	Không chỉ vẽ bounding box. Người 1 phải xuất class_id + confidence + pose_base + grasp_hint + failure_reason.

Mục tiêu học tập: Sau tài liệu này, Người 1 có thể tự học, lấy RGB-D, kiểm tra depth, detect/classify 4 workpieces, tính centroid/yaw/pose_base, tạo grasp_hint và xuất JSON đúng schema để các bạn khác dùng trực tiếp.

Mục lục định hướng

1. Vai trò của Người 1 trong Task 1
2. Các module cần tự học
3. Cấu trúc thư mục và artifact cần nộp
4. Step-by-step triển khai Camera/RGB-D + Perception
5. Confidence score và failure reason
6. Kế hoạch 5 ngày cho Người 1
7. Cách phối hợp với Người 2, Người 3, Người 4 và PM
8. Debug cookbook
9. Definition of Done
10. Gói nộp cá nhân và rubric 100 điểm
11. Phụ lục code skeleton đầy đủ

1. Vai trò của Người 1 trong Task 1

Người 1 phụ trách toàn bộ phần Camera/RGB-D + Perception. Nhiệm vụ không dừng ở việc detect vật trong ảnh, mà phải tạo dữ liệu có cấu trúc để Planner/FSM và Motion Primitive dùng được ngay.

RGB-D camera

- detect object
- classify A/B
- estimate centroid/yaw
- convert pixel/depth to 3D
- convert camera frame to robot base frame
- output object state with confidence and grasp_hint

Câu hỏi Người 1 phải trả lời: Trong scene hiện tại có những vật nào, thuộc loại gì, nằm ở đâu trong robot/base frame, và có đủ tin cậy để gấp không?

Thành phần output

Ý nghĩa

Người dùng output

class_id	Loại vật: part_A hoặc part_B	Người 2 map sang bin A/B
confidence	Độ tin cậy detection/class/pose	Người 2 quyết định gấp hoặc bỏ qua
bbox_xyxy/mask	Vùng vật thể trong ảnh	Người 4 đánh giá perception và debug overlay
centroid_px	Tâm vật trong ảnh	Người 1 dùng để lấy depth
centroid_camera_m	Tâm vật trong camera frame	Người 2 kiểm tra transform
pose_base	Pose vật trong robot/base frame	Người 3 dùng để tạo waypoint gấp
grasp_hint	Gợi ý yaw, grasp width, approach axis	Người 3 dùng trong pick-place primitive
failure_reason	Lý do không nên gấp nếu perception không chắc	Người 2/4 dùng để retry hoặc log lỗi

```

{
  "frame_id": 25,
  "timestamp": 12.53,
  "camera_name": "front_rgbd",
  "objects": [
    {
      "object_id": "obj_001",
      "class_id": "part_A",
      "confidence": 0.91,
      "bbox_xyxy": [310, 220, 380, 292],
      "centroid_px": [345.0, 256.0],
      "centroid_camera_m": [0.12, -0.04, 0.74],
      "pose_base": {
        "position_m": [0.46, -0.15, 0.82],
        "quaternion_xyzw": [0.0, 0.0, 0.707, 0.707]
      },
      "grasp_hint": {
        "yaw_rad": 1.57,
        "grasp_width_m": 0.045,
        "approach_axis": "z_down"
      },
      "failure_reason": null
    }
  ]
}

```

2. Các module Người 1 cần tự học

Thứ tự	Module	Mục cần học	Output học xong
1	Module 1.A - Bắt đầu với Isaac Sim 5.1+	GUI, camera, Stage, Properties, Viewport, Console/Log	Biết mở scene, nhìn camera, đọc log
2	Module 2 - Isaac Sim/USD/URDF/Walker S2/Baseline	Camera path, object path, bin path, scene/config Task 1	task1_perception_start_checklist.md
3	Module 3 - Robotics Math	Frame, transform, quaternion ở mức dùng được	transform_sanity_report.md
4	Module 4 - Camera, RGB-D, Coordinate Transform	Intrinsic, extrinsic, pixel-to-3D, depth sanity, camera-to-base	camera_config_sheet.csv, pixel-to-3D tests
5	Module 5 - Perception for Industrial Manipulation	Detection, segmentation, centroid, yaw, output contract	perception_interface.json, overlays
6	Module 10 - Evaluation, Logging, Ablation	Perception metrics, failure reason, confidence	failure_cases_perception.jsonl

Nguyên tắc tự học: Mỗi phần học xong phải có artifact đo được: ảnh, CSV, JSON, report hoặc unit test. Không đánh dấu hoàn thành chỉ vì đã đọc xong tài liệu.

3. Cấu trúc thư mục và artifact cần nộp

```
src/task1/
  camera_utils.py
  transform_utils.py
  perception.py
  perception_debug.py

configs/
  camera.yaml
  task1_perception.yaml

lab_outputs/perception/
  sample_rgb.png
  sample_depth.npy
  sample_depth_preview.png
  depth_sanity_report.md
  camera_config_sheet.csv
  overlay_detection.png
  overlay_mask.png
  overlay_centroid.png
  confusion_matrix_task1.csv
  pose_estimator_report.csv
  yaw_report.csv
  transform_sanity_report.md
  perception_interface.json
  failure_cases_perception.jsonl
```

Artifact	Bắt buộc?	Ý nghĩa
sample_rgb.png	Có	Chứng minh camera RGB hoạt động và thấy đủ 4 vật
sample_depth.npy	Có	Dữ liệu depth gốc để tính 3D
sample_depth_preview.png	Có	Ảnh visualize depth để kiểm tra nhanh
depth_sanity_report.md	Có	Kiểm tra shape, valid ratio, unit, min/median/max depth
camera_config_sheet.csv	Có	Lưu intrinsics/extrinsics/depth unit
overlay_detection.png	Có	Debug detection/class bằng hình ảnh
confusion_matrix_task1.csv	Nên có	Đánh giá class A/B
pose_estimator_report.csv	Có	Centroid pixel, depth, camera point, base point
yaw_report.csv	Có	Yaw/grasp direction theo từng object
perception_interface.json	Có	Output chính cho Người 2/3/4

4. Step-by-step triển khai Camera/RGB-D + Perception

Step 1. Nhận thông tin scene/config từ PM

PM sẽ hỗ trợ baseline/config, nhưng Người 1 phải tự xác nhận lại thông tin phục vụ perception để tránh làm sai camera hoặc sai object class.

Cần biết	Câu hỏi cần hỏi PM	Ghi chú
Scene Task 1	Scene/USD/config nào đang được dùng?	Ghi rõ file/path
Camera	Camera RGB-D nào dùng cho perception?	Tên camera và prim path
Depth source	Depth trả về mét, mm hay normalized?	Quan trọng để tránh sai scale 1000 lần

Object names	Workpieces có tên/path/label như thế nào?	Dùng để debug hoặc đối chiếu
Class A/B	Phân biệt A/B bằng màu, shape, semantic label hay config?	Quyết định cách classify
Bin A/B	Bin A/B nằm ở đâu?	Người 2 dùng để map class -> bin
Transform	Có sẵn T_base_camera không?	Nếu chưa có, phối hợp Người 2

```
# task1_perception_start_checklist.md
- Scene/config: ...
- Camera name/path: ...
- RGB resolution: ...
- Depth resolution: ...
- Depth unit: unknown / meter / millimeter
- Object naming pattern: ...
- Class mapping rule: ...
- Bin A/B info source: ...
- T_base_camera source: ...
- Open questions:
  1. ...
  2. ...
```

Step 2. Lấy RGB/depth sample đầu tiên

Mục tiêu của bước này là chứng minh camera hoạt động, ảnh nhìn thấy đủ 4 vật và depth có dữ liệu hợp lệ.

Output	Yêu cầu đạt
sample_rgb.png	Ảnh không đen, thấy bàn và đủ workpieces
sample_depth.npy	Mảng depth gốc, lưu đúng dtype
sample_depth_preview.png	Depth được normalize để xem bằng mắt
depth_sanity_report.md	Có shape, valid_ratio, min/median/max, nhận định depth unit

```
import numpy as np

depth = np.load("sample_depth.npy")
valid = np.isfinite(depth) & (depth > 0)
print("shape:", depth.shape)
print("valid_ratio:", valid.mean())

if valid.any():
    print("min:", depth[valid].min())
    print("median:", np.median(depth[valid]))
    print("max:", depth[valid].max())
else:
    print("No valid depth values")
```

Cách đoán depth unit: Nếu median depth khoảng 0.3-2.0 thì thường là mét. Nếu khoảng 300-2000 thì có thể là millimeter và cần chia 1000 trước khi tính 3D.

Step 3. Tạo camera_config_sheet.csv

File này giúp cả nhóm không phải đoán camera intrinsics/extrinsics mỗi lần debug.

```
camera_name,width,height,fx,fy,cx,cy,depth_unit,T_base_camera_source,notes
front_rgbd,640,480,615.2,615.0,320.0,240.0,meter,scene_or_config,"Task 1 main camera"
```

Thông số	Ý nghĩa	Lỗi thường gặp
fx, fy	Tiêu cự theo pixel	Dùng sai fx/fy làm point cloud méo
cx, cy	Optical center	Centroid 3D lệch nếu principal point sai
width, height	Kích thước ảnh	RGB/depth không cùng resolution

depth_unit	Đơn vị depth	Sai scale 1000 lần
T_base_camera_source	Nguồn camera-to-base transform	Không biết transform lấy từ đâu thì khó debug

Step 4. Viết hàm pixel-to-3D

Từ centroid pixel và depth, tính điểm 3D trong camera frame. Đây là công thức quan trọng nhất của phần RGB-D.

```
import numpy as np

def pixel_to_camera_point(u, v, depth_z, fx, fy, cx, cy):
    """
    Convert pixel + depth to a 3D point in camera frame.
    u, v: pixel coordinate
    depth_z: depth value in meters
    fx, fy, cx, cy: camera intrinsics
    """
    if depth_z <= 0 or not np.isfinite(depth_z):
        raise ValueError(f"Invalid depth value: {depth_z}")

    x = (u - cx) * depth_z / fx
    y = (v - cy) * depth_z / fy
    z = depth_z
    return np.array([x, y, z], dtype=float)

def test_pixel_center():
    p = pixel_to_camera_point(
        u=320, v=240, depth_z=1.0,
        fx=600, fy=600, cx=320, cy=240
    )
    assert np.allclose(p, [0.0, 0.0, 1.0])
```

Step 5. Lấy depth ổn định quanh centroid

Không nên lấy depth đúng 1 pixel vì dễ nhiễu hoặc rơi vào biên object. Dùng median trong patch nhỏ hoặc trong mask.

```
def robust_depth_from_patch(depth, u, v, radius=3):
    h, w = depth.shape
    u = int(round(u))
    v = int(round(v))

    x1 = max(0, u - radius)
    x2 = min(w, u + radius + 1)
    y1 = max(0, v - radius)
    y2 = min(h, v + radius + 1)

    patch = depth[y1:y2, x1:x2]
    valid = patch[np.isfinite(patch) & (patch > 0)]
    if len(valid) == 0:
        return None
    return float(np.median(valid))

def robust_depth_from_mask(depth, mask):
    valid = depth[(mask > 0) & np.isfinite(depth) & (depth > 0)]
    if len(valid) == 0:
        return None
    return float(np.median(valid))
```

Step 6. Detect vật bằng baseline đơn giản trước

Mục tiêu ban đầu là có MVP end-to-end, không phải model đẹp. Nếu màu/hình dạng trong mô phỏng rõ, bắt đầu bằng threshold + contour.

Mức	Cách làm	Khi nào dùng
Level 0	Threshold màu/depth + contour	Làm MVP nhanh
Level 1	Mask + contour + minAreaRect/PCA	Khi cần centroid/yaw tốt hơn
Level 2	YOLO/segmentation model	Khi rule-based không đủ ổn định

```
import cv2 as cv
import numpy as np

def detect_by_color_bgr(rgb_bgr, lower_hsv, upper_hsv):
    hsv = cv.cvtColor(rgb_bgr, cv.COLOR_BGR2HSV)
    mask = cv.inRange(hsv, np.array(lower_hsv), np.array(upper_hsv))

    kernel = np.ones((5, 5), np.uint8)
    mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
    mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)

    contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
    detections = []
    for cnt in contours:
        area = cv.contourArea(cnt)
        if area < 100:
            continue
        x, y, w, h = cv.boundingRect(cnt)
        m = cv.moments(cnt)
        if m["m00"] == 0:
            continue
        cx = m["m10"] / m["m00"]
        cy = m["m01"] / m["m00"]
        detections.append({
            "bbox_xyxy": [x, y, x + w, y + h],
            "centroid_px": [cx, cy],
            "area_px": area,
            "contour": cnt
        })
    return detections, mask

# Nếu hai loại vật khác màu
# detections_A, mask_A = detect_by_color_bgr(rgb, lower_A, upper_A)
# detections_B, mask_B = detect_by_color_bgr(rgb, lower_B, upper_B)

for d in detections_A:
    d["class_id"] = "part_A"
    d["confidence"] = 0.85

for d in detections_B:
    d["class_id"] = "part_B"
    d["confidence"] = 0.85
```

Step 7. Vẽ overlay để debug bằng hình ảnh

Quy tắc: Không được nói “detection đúng” nếu chưa có overlay. Overlay là bằng chứng để cả nhóm biết lỗi nằm ở perception hay motion.

```
def draw_detections(rgb_bgr, objects):
    img = rgb_bgr.copy()
    for obj in objects:
        x1, y1, x2, y2 = obj["bbox_xyxy"]
        u, v = obj["centroid_px"]
        label = f'{obj["class_id"]} {obj["confidence"]:.2f}'

        cv.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

```

cv.circle(img, (int(u), int(v)), 4, (0, 0, 255), -1)
cv.putText(img, label, (x1, max(0, y1 - 5)),
           cv.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
return img

```

Step 8. Tính yaw/grasp direction

Mức	Cách tính yaw	Khi nào dùng
MVP	yaw = 0 hoặc yaw default	Nếu gripper gấp được vật không cần xoay
Khá	cv.minAreaRect(contour)	Nếu vật hình chữ nhật/dài
Tốt	PCA trên mask/point cloud	Nếu cần trực chính ổn định

```

def estimate_yaw_from_contour(contour):
    rect = cv.minAreaRect(contour)
    angle_deg = rect[-1]
    # OpenCV angle convention can be tricky.
    # Treat this as a grasp hint and validate visually.
    yaw_rad = np.deg2rad(angle_deg)
    return float(yaw_rad)

```

Step 9. Tính centroid 3D trong camera frame

```

u, v = detection["centroid_px"]
z = robust_depth_from_patch(depth, u, v, radius=3)

if z is None:
    detection["failure_reason"] = "INVALID_DEPTH"
else:
    p_cam = pixel_to_camera_point(u, v, z, fx, fy, cx, cy)

# pose_estimator_report.csv columns
object_id,class_id,u,v,depth_m,x_cam,y_cam,z_cam,status
obj_001,part_A,345,256,0.74,0.031,0.019,0.740,ok

```

Step 10. Chuyển centroid từ camera frame sang base frame

```

def transform_point(T_ab, p_b):
    p_h = np.array([p_b[0], p_b[1], p_b[2], 1.0])
    return (T_ab @ p_h)[:3]

# Quy ước dùng trong nhóm:
# p_base = T_base_camera @ p_camera

```

Sanity test	Kỳ vọng
Object trên bàn có z hợp lý	z_base gần mặt bàn/object height
Vật bên trái/phải ảnh	Không bị đảo trái/phải bất thường
Pose object nằm trong workspace	Nếu quá xa, có thể sai transform/depth scale
T identity test	Không làm đổi điểm

Step 11. Tạo ObjectState hoàn chỉnh

```

{
  "frame_id": 1,
  "timestamp": 0.033,
  "camera_name": "front_rgb",
  "objects": [
    {
      "object_id": "obj_001",
      "class_id": "part_A",
      "confidence": 0.88,
      "bbox_xyxy": [310, 220, 380, 292],
      "centroid_px": [345.0, 256.0],

```

```

    "centroid_camera_m": [0.031, 0.019, 0.740],
    "pose_base": {
        "position_m": [0.46, -0.15, 0.82],
        "quaternion_xyzw": [0.0, 0.0, 0.707, 0.707]
    },
    "grasp_hint": {
        "approach_axis": "z_down",
        "yaw_rad": 1.57,
        "grasp_width_m": 0.045
    },
    "failure_reason": null
}
],
"summary": {
    "num_objects": 4,
    "num_valid_objects": 4,
    "num_invalid_depth": 0
}
}

```

5. Confidence score và failure reason

Confidence ở giai đoạn đầu không cần quá phức tạp, nhưng phải có ý nghĩa để Planner biết nên gấp hay bỏ qua.

```

confidence = 1.0
if area_px < min_area:
    confidence -= 0.3
if depth_invalid:
    confidence -= 0.4
if mask_quality_low:
    confidence -= 0.2
if class_ambiguous:
    confidence -= 0.2
confidence = max(0.0, min(1.0, confidence))

```

Trường hợp	Confidence gợi ý
Mask rõ, depth tốt, class chắc	0.85-0.95
Mask hơi nhiễu nhưng centroid ổn	0.70-0.85
Depth thiếu hoặc class không chắc	< 0.70
Không đủ thông tin để gấp	< 0.50

Failure reason	Khi nào dùng
NO_OBJECT_DETECTED	Không detect đủ vật
LOW_CONFIDENCE	Detect được nhưng không đủ tin cậy
INVALID_DEPTH	Depth tại object không hợp lệ
CLASS_AMBIGUOUS	Không chắc A/B
MASK_TOO_SMALL	Mask nhỏ hơn ngưỡng
MASK_FRAGMENTED	Mask bị vỡ thành nhiều mảnh
POSE_OUT_OF_RANGE	Pose 3D bất thường
TRANSFORM_NOT_AVAILABLE	Chưa có T_base_camera
GRASP_HINT_UNSTABLE	Yaw/grasp direction dao động mạnh

6. Kế hoạch 5 ngày cho Người 1

Ngày	Mục tiêu	Output bắt buộc	Checklist pass
1	Camera và dữ liệu mẫu	sample_rgb.png, sample_depth.npy, sample_depth_preview.png,	RGB thấy đủ 4 vật; depth valid >80%; biết depth unit

		camera_config_sheet.csv, depth_sanity_report.md	
2	Pixel-to-3D và transform sanity	camera_utils.py, transform_utils.py, pytest_camera_utils.txt, pose_estimator_report.csv, transform_sanity_report.md	Test optical center pass; p_camera/p_base hợp lý
3	Detection/classification MVP	overlay_detection.png, overlay_mask.png, confusion_matrix_task1.csv, perception_mvp_report.md	Detect đủ 4 vật, class A/B đúng ở sample
4	Pose/yaw/grasp hint	pose_estimator_report.csv, yaw_report.csv, perception_interface.json, overlay_centroid.png	Mỗi vật có pose_base và grasp_hint
5	Robustness và tích hợp	perception_eval_summary.csv, failure_cases_perception.jsonl, perception_interface_v1.json	Chạy nhiều frame/seed; Người 2 đọc được JSON

7. Cách phối hợp với Người 2, Người 3, Người 4 và PM

7.1. Với Người 2 - Transform + Planner/FSM

Câu hỏi cần thống nhất	Quyết định đề xuất
Planner cần class tên gì?	part_A, part_B
Pose ở frame nào?	pose_base
Confidence threshold?	0.70
Nếu depth invalid thì làm gì?	failure_reason = INVALID_DEPTH
Object ID có ổn định không?	obj_001... hoặc ID theo contour/track

7.2. Với Người 3 - Motion Primitive

Câu hỏi cần hỏi	Vì sao
Gripper cần yaw chính xác đến mức nào?	Đề biết yaw MVP có đủ không
Cần grasp_width không?	Nếu cần, ước lượng từ bbox/mask
Approach axis mặc định là gì?	Thường dùng z_down
Pose cần ở tâm vật hay mép vật?	Tùy hình dạng vật và gripper

7.3. Với Người 4 - Evaluation/Debug

File	Người 4 dùng để làm gì
confusion_matrix_task1.csv	Đánh giá phân loại A/B
pose_estimator_report.csv	Phân tích pose sai
failure_cases_perception.jsonl	Thống kê lỗi perception
overlay images	Debug bằng chứng trực quan

7.4. Với PM

Người 1 cần PM hỗ trợ	Không nên đẩy cho PM
Scene/config/camera path ban đầu	Viết detection/perception thay Người 1
Cách chạy script baseline	Tính centroid, yaw, depth sanity
Lỗi môi trường, asset path, camera không xuất dữ liệu	Làm perception report

8. Debug cookbook

Lỗi	Triệu chứng	Cách kiểm tra	Cách sửa
RGB không thấy vật	Overlay trống	Kiểm tra camera view/path	Hỏi PM camera đúng chưa, chỉnh camera/config
Depth toàn 0/NaN	Không tính 3D được	depth_sanity_report	Kiểm tra depth annotator/source
Depth sai đơn vị	Điểm 3D quá xa/gần	Median depth bất thường	Đổi mm -> m hoặc scale đúng
Mask nhiều	Detect quá nhiều object	overlay_mask.png	Morphology open/close, lọc area
Detect thiếu vật	Chỉ thấy 2-3 object	Contour area/threshold	Chỉnh threshold, dùng depth/mask
Class sai	Vật vào sai bin	confusion_matrix_task1.csv	Rule mới, crop classifier, nhiều frame
Centroid sai	Robot gấp lệch	overlay_centroid.png	Dùng mask tốt hơn, median depth theo mask
Yaw sai	Gripper xoay lệch	Overlay trục yaw	Thử yaw + $\pi/2$, PCA/minAreaRect
pose_base sai	Tay đi sai hướng	transform_sanity_report	Kiểm tra T_base_camera, convention, quaternion
Confidence quá cao dù sai	Planner gấp vật lỗi	failure_cases	Thêm penalty cho mask/depth/class ambiguity

9. Definition of Done cho Người 1

Điều kiện	Bắt buộc
Có sample_rgb.png, sample_depth.npy	Có
Có depth_sanity_report.md	Có
Có camera_config_sheet.csv	Có
Detect đủ 4 workpieces ở scene mẫu	Có
Classify A/B đúng ở scene mẫu	Có
Tính được centroid pixel và centroid 3D	Có
Xuất được pose_base	Có
Có grasp_hint	Có
Có perception_interface.json đúng schema	Có
Có overlay debug	Có
Có failure reason rõ	Có
Người 2 đọc được JSON và chọn được object	Có
Người 3 dùng được pose/grasp hint để dry-run	Có
Người 4 có report để đánh giá perception	Có

10. Gói nộp cá nhân và rubric 100 điểm

```

person1_perception_pack/
  README_PERCEPTION.md
  src/
    camera_utils.py
    transform_utils.py
    perception.py
    perception_debug.py
  configs/
    task1_perception.yaml
    camera.yaml
  outputs/

```

```

sample_rgb.png
sample_depth.npy
sample_depth_preview.png
camera_config_sheet.csv
depth_sanity_report.md
overlay_detection.png
overlay_mask.png
overlay_centroid.png
confusion_matrix_task1.csv
pose_estimator_report.csv
yaw_report.csv
perception_interface.json
failure_cases_perception.jsonl

```

```

# README_PERCEPTION.md
## What it does
Detects 4 workpieces, classifies A/B, estimates 3D centroid and grasp hint.

## Inputs
- RGB image
- Depth image
- Camera intrinsics
- T_base_camera

## Outputs
- perception_interface.json

## Known limitations
- ...

```

Hạng mục	Điểm
Lấy RGB/depth và camera config đúng	15
Depth sanity và pixel-to-3D đúng	15
Detection đủ 4 vật	15
Classification A/B	15
Centroid 3D và pose_base	15
Yaw/grasp hint	10
JSON interface đúng schema	10
Overlay/debug/failure report rõ	5

11. Phụ lục code skeleton đầy đủ

11.1. camera_utils.py

```

import numpy as np

class CameraIntrinsics:
    def __init__(self, fx, fy, cx, cy, width=None, height=None):
        self.fx = float(fx)
        self.fy = float(fy)
        self.cx = float(cx)
        self.cy = float(cy)
        self.width = width
        self.height = height

def pixel_to_camera_point(u, v, depth_z, intr):
    if depth_z is None or depth_z <= 0 or not np.isfinite(depth_z):
        raise ValueError(f"Invalid depth: {depth_z}")
    x = (u - intr.cx) * depth_z / intr.fx

```

```

y = (v - intr.cy) * depth_z / intr.fy
z = depth_z
return np.array([x, y, z], dtype=float)

def robust_depth_from_patch(depth, u, v, radius=3):
    h, w = depth.shape
    u = int(round(u))
    v = int(round(v))
    x1, x2 = max(0, u-radius), min(w, u+radius+1)
    y1, y2 = max(0, v-radius), min(h, v+radius+1)
    patch = depth[y1:y2, x1:x2]
    valid = patch[np.isfinite(patch) & (patch > 0)]
    return None if len(valid) == 0 else float(np.median(valid))

```

11.2. perception.py

```

import cv2 as cv
import numpy as np
from camera_utils import pixel_to_camera_point, robust_depth_from_patch

def detect_by_color_bgr(rgb_bgr, lower_hsv, upper_hsv, min_area=100):
    hsv = cv.cvtColor(rgb_bgr, cv.COLOR_BGR2HSV)
    mask = cv.inRange(hsv, np.array(lower_hsv), np.array(upper_hsv))
    kernel = np.ones((5, 5), np.uint8)
    mask = cv.morphologyEx(mask, cv.MORPH_OPEN, kernel)
    mask = cv.morphologyEx(mask, cv.MORPH_CLOSE, kernel)
    contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
    detections = []
    for cnt in contours:
        area = cv.contourArea(cnt)
        if area < min_area:
            continue
        x, y, w, h = cv.boundingRect(cnt)
        m = cv.moments(cnt)
        if m["m00"] == 0:
            continue
        cx = m["m10"] / m["m00"]
        cy = m["m01"] / m["m00"]
        detections.append({
            "bbox_xyxy": [int(x), int(y), int(x+w), int(y+h)],
            "centroid_px": [float(cx), float(cy)],
            "area_px": float(area),
            "contour": cnt,
        })
    return detections, mask

def estimate_yaw_from_contour(contour):
    rect = cv.minAreaRect(contour)
    return float(np.deg2rad(rect[-1]))

def make_object_state(d, class_id, depth, intr, T_base_camera):
    u, v = d["centroid_px"]
    z = robust_depth_from_patch(depth, u, v, radius=3)
    if z is None:
        return {"failure_reason": "INVALID_DEPTH", **d}
    p_cam = pixel_to_camera_point(u, v, z, intr)
    p_h = np.array([p_cam[0], p_cam[1], p_cam[2], 1.0])
    p_base = (T_base_camera @ p_h)[:3]
    yaw = estimate_yaw_from_contour(d["contour"])
    return {
        "object_id": None,
        "class_id": class_id,

```

```

    "confidence": 0.85,
    "bbox_xyxy": d["bbox_xyxy"],
    "centroid_px": d["centroid_px"],
    "centroid_camera_m": p_cam.tolist(),
    "pose_base": {
        "position_m": p_base.tolist(),
        "quaternion_xyzw": [0.0, 0.0, 0.0, 1.0]
    },
    "grasp_hint": {
        "approach_axis": "z_down",
        "yaw_rad": yaw,
        "grasp_width_m": 0.045
    },
    "failure_reason": None
}

```

Kết luận: Người 1 thành công khi xuất được object state đủ sạch để robot gấp: class_id + confidence + pose_base + grasp_hint + failure_reason. Đây là đầu vào quan trọng nhất cho Planner/FSM và Motion Primitive trong Task 1.